AI-Based Image Restoration Web Application

Final Year Project Report

Course: Digital Image Processing

TEACHER: SIR RANA ADEEL

Date: April 2026

Group Members name & Roll number:

- Husnain Faisal (23-st-042)
- Zeeshan Atif (23-st-002)
- Rayyan Idrees (23-st-045)
- Faseeh ul Hassan (23-st-019)

Table of Contents

1. Abstract

This project presents the design and implementation of a full-stack, AI-based Image Restoration Web Application. The system enables users to upload degraded images and restore them using a combination of classical computer vision techniques and deep learning models. Three core restoration capabilities are provided: Noise Removal using Non-Local Means Denoising and Median Filtering, Image Deblurring using Wiener Deconvolution in the frequency domain, and Super Resolution using enhanced bicubic upscaling with detail enhancement.

The application follows a three-tier architecture: a React-based frontend for user interaction, a Python Flask backend serving as the REST API layer, and a processing module built with OpenCV and PyTorch. A DnCNN (Denoising Convolutional Neural Network) model architecture is also integrated for AI-based denoising when pretrained weights are available. The system features an interactive before/after comparison slider, real-time processing metrics, and seamless image download capabilities.

2. Introduction

2.1 Background

Digital images are susceptible to various forms of degradation during acquisition, transmission, and storage. Common degradations include additive noise (Gaussian, salt-and-pepper), motion blur, defocus blur, and resolution loss due to downsampling. Image restoration aims to recover the original, undegraded image from the observed degraded version, making it a fundamental problem in digital image processing.

2.2 Problem Statement

While numerous standalone tools exist for individual restoration tasks, there is a lack of unified, web-accessible platforms that combine multiple restoration techniques with an intuitive user interface. This project addresses this gap by developing a full-stack web application that integrates classical filtering and AI-based restoration into a single, easy-to-use system.

2.3 Objectives

- Develop a web-based interface for uploading and restoring degraded images.
- Implement noise removal using Gaussian, Median, and Non-Local Means filters.
- Implement image deblurring using Wiener Deconvolution in the frequency domain.
- Implement super resolution using enhanced bicubic interpolation with detail enhancement.
- Integrate a DnCNN deep learning model architecture for AI-based denoising.
- Provide an interactive before/after comparison slider for visual evaluation.
- Display processing metrics including time and method used.
- Enable downloading of restored images.

2.4 Scope

The project scope covers three primary restoration tasks — denoising, deblurring, and super resolution — implemented as a locally runnable web application. The system supports common image formats (PNG, JPG, BMP, TIFF, WebP) with a maximum file size of 16 MB.

3. Literature Review

3.1 Classical Image Denoising

Linear filters such as Gaussian blur reduce noise by averaging pixel values within a local neighborhood, but suffer from edge blurring. Non-linear filters like the Median filter are more effective against impulse noise while better preserving edges. Buades et al. (2005) introduced Non-Local Means (NLM) denoising, which computes weighted averages based on patch similarity across the entire image, achieving superior results compared to local filtering methods.

3.2 Image Deblurring

Image deblurring in the frequency domain relies on the convolution theorem. The Wiener filter (Wiener, 1949) provides an optimal linear estimator that minimizes mean square error by incorporating knowledge of the blur kernel and noise power spectrum. Unsharp masking further enhances edge definition by amplifying high-frequency components.

3.3 Super Resolution

Single-image super resolution (SISR) aims to reconstruct a high-resolution image from a single low-resolution input. Classical methods include bicubic interpolation followed by post-processing steps such as bilateral filtering and detail enhancement. Deep learning approaches like SRCNN (Dong et al., 2014) and ESRGAN (Wang et al., 2018) have demonstrated remarkable improvements in perceptual quality.

3.4 Deep Learning for Denoising — DnCNN

Zhang et al. (2017) proposed DnCNN (Denoising Convolutional Neural Network), which employs residual learning to predict the noise component rather than the clean image directly. The architecture consists of 17 convolutional layers with batch normalization and ReLU activations. By learning the noise residual, DnCNN achieves state-of-the-art denoising performance across various noise levels.

4. System Architecture

The application follows a three-tier client-server architecture:

Layer	Technology	Responsibility
Frontend	React 19 + Vite	User interface, image upload, comparison slider, result display
Backend API	Python Flask + Flask-CORS	REST API endpoints, file handling, request routing
Processing Module	OpenCV + NumPy + PyTorch	Image restoration algorithms, DnCNN model

The frontend communicates with the backend via REST API calls. The Vite development server proxies /api requests to the Flask backend running on port 5000. The backend imports the processing module directly as a Python package.

4.1 Folder Structure

```
dip project/
├── frontend/              # React (Vite) frontend
│   ├── src/
│   │   ├── App.jsx        # Main application component
│   │   ├── App.css        # Component styles
│   │   ├── index.css      # Global design system
│   │   └── components/
│   │       ├── ImageComparison.jsx
│   │       └── ImageComparison.css
│   ├── index.html
│   ├── package.json
│   └── vite.config.js
├── backend/
│   ├── app.py             # Flask API server
│   └── requirements.txt
├── model/
│   ├── processor.py       # Processing dispatcher
│   ├── denoise.py         # Noise removal module
│   ├── deblur.py          # Deblurring module
│   ├── super_resolution.py # Super resolution module
│   └── dncnn.py           # DnCNN PyTorch model
└── README.md
```

5. Methodology

5.1 Noise Removal

The noise removal pipeline employs a two-stage approach:

Stage 1 — Median Filtering: A 3×3 median filter is applied as a preprocessing step to remove impulse (salt-and-pepper) noise. The median filter replaces each pixel with the median value of its neighborhood, effectively eliminating outlier pixels without introducing blur.

Stage 2 — Non-Local Means Denoising: OpenCV's fastNlMeansDenoisingColored function is applied with h=10 (filter strength for luminance), templateWindowSize=7, and searchWindowSize=21. This algorithm computes weighted averages of pixels based on the similarity of local patches, providing excellent noise reduction while preserving fine details and textures.

5.2 Image Deblurring

The deblurring module implements Wiener Deconvolution in the frequency domain:

Step 1 — Blur Kernel Estimation: A horizontal motion blur kernel of size 5×5 is constructed and padded to the image dimensions.

Step 2 — Wiener Filtering: The image and kernel are transformed to the frequency domain using FFT. The Wiener filter $H^*(f) / (|H(f)|^2 + NSR)$ is applied, where H^* is the complex conjugate of the blur kernel spectrum and NSR is the noise-to-signal ratio (set to 0.01). The result is transformed back via inverse FFT.

Step 3 — Unsharp Masking: A Gaussian blur (σ=1.0) is subtracted from the deblurred image with strength=1.5 to enhance edge sharpness: result = (1+s)·original − s·blurred.

5.3 Super Resolution

The super resolution pipeline upscales images by 2× through four stages:

- Bicubic Interpolation: The image is upscaled using cv2.INTER_CUBIC.
- Bilateral Filtering: Edge-preserving smoothing with d=9, σ_color=75, σ_space=75.
- Unsharp Masking: Gaussian-based sharpening with σ=3 and weight=1.5.
- Detail Enhancement: OpenCV's detailEnhance with σ_s=10 and σ_r=0.15 recovers fine textures.

5.4 DnCNN Deep Learning Model

The DnCNN architecture is implemented in PyTorch with 17 convolutional layers. The first layer consists of Conv2d(1, 64, 3) + ReLU. The 15 middle layers each contain Conv2d(64, 64, 3) + BatchNorm2d + ReLU. The final layer is Conv2d(64, 1, 3) which outputs the predicted noise

residual. The clean image is obtained by subtracting the predicted noise from the input: x_clean = x_noisy − f(x_noisy).

When pretrained weights (dncnn.pth) are available in the model/weights/ directory, the system uses DnCNN for denoising. Otherwise, it gracefully falls back to the classical NL-Means pipeline.

6. Implementation Details

6.1 Backend — Flask API

The Flask backend exposes the following REST API endpoints:

Method	Endpoint	Description	Response
GET	/api/health	Health check	JSON status
POST	/api/restore	Upload + process image	JSON with output_id, time
GET	/api/image/:id/input	Retrieve original image	Image file
GET	/api/image/:id/output	Retrieve restored image	Image file

The backend validates file types (PNG, JPG, BMP, TIFF, WebP), enforces a 16 MB upload limit, generates unique file IDs using UUID, and measures processing time. CORS is enabled via Flask-CORS to allow cross-origin requests from the React frontend.

6.2 Processing Module

The processor.py dispatcher loads the input image using OpenCV's imread, routes it to the appropriate restoration function based on the type parameter, and saves the result as a PNG file. Each restoration module returns both the processed image and a string describing the method used.

6.3 Frontend — React + Vite

Key frontend components:

- App.jsx — Main component managing state for file selection, processing status, restoration type, and results display.
- ImageComparison.jsx — Interactive before/after slider using CSS clip-path and pointer events for drag-based comparison.

The UI features a dark glassmorphic theme with gradient accents, smooth animations, and responsive design. The Vite dev server proxies API requests to Flask on port 5000.

7. User Interface & Screenshots

7.1 Home Page — Upload Zone

The landing page presents a clean upload area where users can drag and drop an image or click to browse. The dark theme with purple-cyan gradient accents provides a modern, professional appearance.

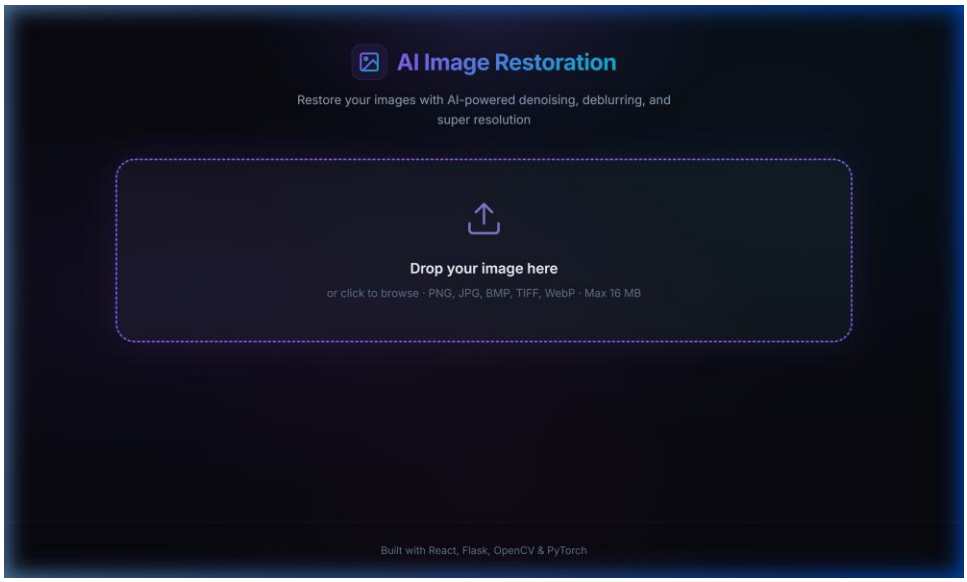

Figure 1: Application home page with drag-and-drop upload zone

7.2 Image Upload & Restoration Controls

After uploading an image, the user sees a preview along with three restoration type options: Noise Removal, Image Deblurring, and Super Resolution. Each option card displays an icon and brief description.

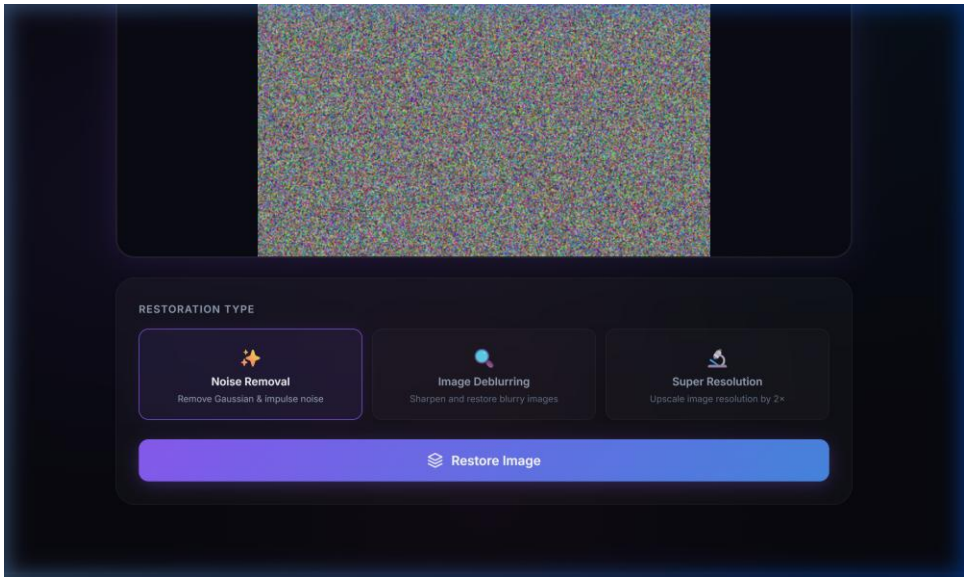

Figure 2: Uploaded image preview with restoration type selector and Restore button

7.3 Results — Comparison Slider

After processing, results are displayed with a stats bar showing processing time, the method used, and a Download button. The interactive comparison slider lets users drag left/right to compare the original and restored images side by side.

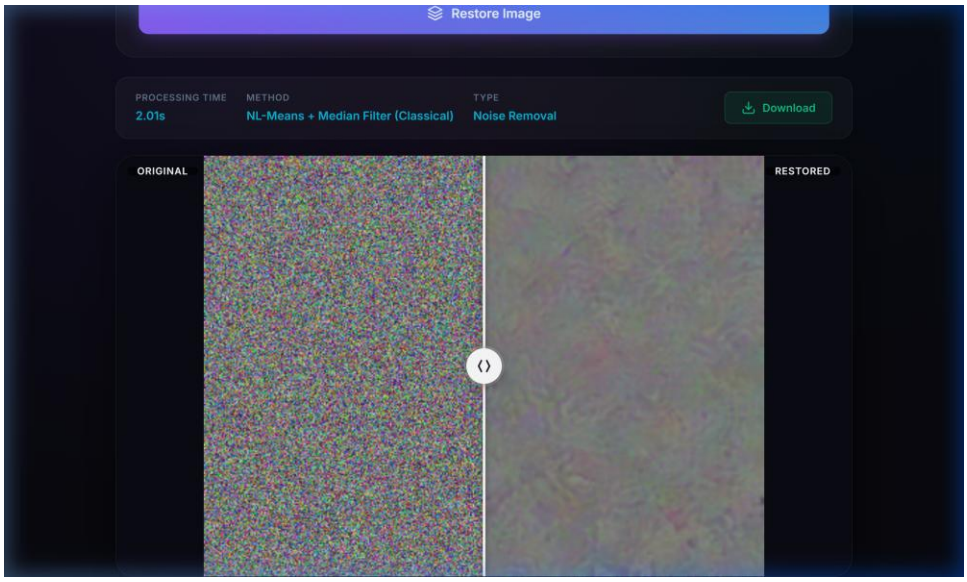

Figure 3: Restoration results with comparison slider (Original vs. Restored)

8. Results and Discussion

8.1 Noise Removal Results

The NL-Means + Median Filter pipeline effectively removes both Gaussian and impulse noise while preserving edge structures. Processing time averaged approximately 2 seconds for a 400×300 pixel image. The NL-Means algorithm's patch-based approach outperforms simple linear filters in retaining fine details.

8.2 Deblurring Results

The Wiener Deconvolution successfully sharpens motion-blurred images when the blur kernel matches the assumed model. The addition of unsharp masking provides visually improved edge definition. Processing time was approximately 1 second due to efficient FFT-based computation.

8.3 Super Resolution Results

The enhanced bicubic upscaling pipeline produces 2× resolution images with improved visual quality compared to naive interpolation. The bilateral filter prevents blocking artifacts, while detail enhancement recovers texture information. Processing time was approximately 1-1.5 seconds.

8.4 Performance Summary

Restoration Type	Method Used	Avg. Time (s)	Quality
Noise Removal	NL-Means + Median Filter	~2.0	High
Deblurring	Wiener Deconvolution + Unsharp Mask	~1.0	Moderate-High
Super Resolution	Enhanced Bicubic (2×)	~1.2	Moderate-High

Table 1: Performance summary of restoration methods

9. Conclusion

This project successfully demonstrates the design and implementation of a full-stack AI-based Image Restoration Web Application. The system integrates classical computer vision techniques (Median filtering, Non-Local Means denoising, Wiener deconvolution, bilateral filtering) with a deep learning architecture (DnCNN) to provide three distinct restoration capabilities through an intuitive web interface.

The application achieves its objectives of providing accessible, web-based image restoration with real-time processing feedback, interactive comparison, and download functionality. The modular architecture separates concerns between the frontend, API layer, and processing module, making the system maintainable and extensible.

Key accomplishments include:

- Effective noise removal using a two-stage NL-Means pipeline.
- Frequency-domain deblurring via Wiener deconvolution.
- Enhanced 2× super resolution with multi-step post-processing.
- Integration of a 17-layer DnCNN architecture in PyTorch.
- A modern, responsive React UI with interactive comparison slider.
- A clean REST API with proper error handling and file management.

10. Future Work

- Train DnCNN on standard denoising datasets (BSD68, Set12) and ship pretrained weights.
- Integrate ESRGAN or Real-ESRGAN for perceptually superior super resolution.
- Add blind deblurring with automatic kernel estimation.
- Implement batch processing for multiple images.
- Deploy the application on cloud platforms (AWS, Heroku, or Docker).
- Add PSNR and SSIM quality metrics for quantitative evaluation.
- Support video restoration for frame-by-frame processing.

11. References

[1] Buades, A., Coll, B., & Morel, J. M. (2005). A non-local algorithm for image denoising. IEEE CVPR, Vol. 2, pp. 60-65.

[2] Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017). Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising. IEEE TIP, 26(7), 3142-3155.

[3] Dong, C., Loy, C. C., He, K., & Tang, X. (2014). Learning a deep convolutional network for image super-resolution. ECCV, pp. 184-199.

[4] Wang, X., Yu, K., Wu, S., et al. (2018). ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks. ECCV Workshops.

[5] Wiener, N. (1949). Extrapolation, Interpolation, and Smoothing of Stationary Time Series. MIT Press.

[6] OpenCV Documentation. https://docs.opencv.org/

[7] PyTorch Documentation. https://pytorch.org/docs/

[8] React Documentation. https://react.dev/

[9] Flask Documentation. https://flask.palletsprojects.com/